UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA INFORMÁTICA



PROYECTO PARCIAL

Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Curso:	Programación Orientada a Objetos
Docentes:	MSc. Wilder Nina Choquehuayta (principal) MSc. Percy Lovón Ramos (auxiliar) Dr. Ortiz Ortiz Emanuel (auxiliar)
Tema asignado:	Fuga de la Prisión
Ciclo:	IV
Integrantes:	JOSE FERNANDO CHAVEZ LOZANO MICKELE CARLOS TORRES VELAZCO PIERO CEMEI ROMERO MEZA RICHARD AARON SORIANO CORDOVA
Período académico:	2026-II

Lima — Perú

2026

Informe parcial Fuga de la Prisión

I. Resumen Ejecutivo

Este avance presenta un nivel jugable de «Fuga de la Prisión»: el personaje busca una llave y escapa sin ser detectado. El diseño separa personajes, reglas, estados de la partida, puntaje y dibujo para que cada parte tenga una función clara. Se muestran nueve clases principales en UML y ejemplos de asociación, agregación, composición, encapsulamiento, herencia y polimorfismo. También se aplican los patrones State y Observer. Las capturas, las pruebas y el historial Git muestran el avance disponible.

II. Introducción

a. Contexto y objetivo

El proyecto es una evaluación del curso de Programación Orientada a Objetos mediante un videojuego 2D. La temática asignada es escapar de una prisión evitando guardias, cámaras y sistemas de seguridad [1, p. 2]. En esta entrega parcial se busca demostrar los conceptos de POO y los patrones de diseño en un primer nivel que ya se puede jugar. La rúbrica también pide mostrar el avance del juego, el uso de Git y una explicación oral clara [2, p. 1].

b. Tecnologías utilizadas

El juego está hecho en Python 3.12. Pygame 2.6.1 dibuja la ventana y recibe las teclas. Mermaid.js permite editar el diagrama UML; `unittest` comprueba las reglas del juego; y Git registra las versiones del proyecto.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El diagrama principal muestra nueve clases clave de las dieciocho definidas en el código. La rúbrica pide ocho o más clases bien fundamentadas y un diagrama de las clases principales; no exige dibujar todas [2, pp. 1–2]. Las otras nueve corresponden a estados de la partida, la habilidad de correr, el observador del puntaje y la vista. Cada caja tiene tres partes: nombre, atributos y métodos. El triángulo representa herencia; el rombo negro, composición; el rombo blanco, agregación; la flecha continua, asociación; y la flecha discontinua, dependencia. Los números indican cuántos objetos participan en cada relación.

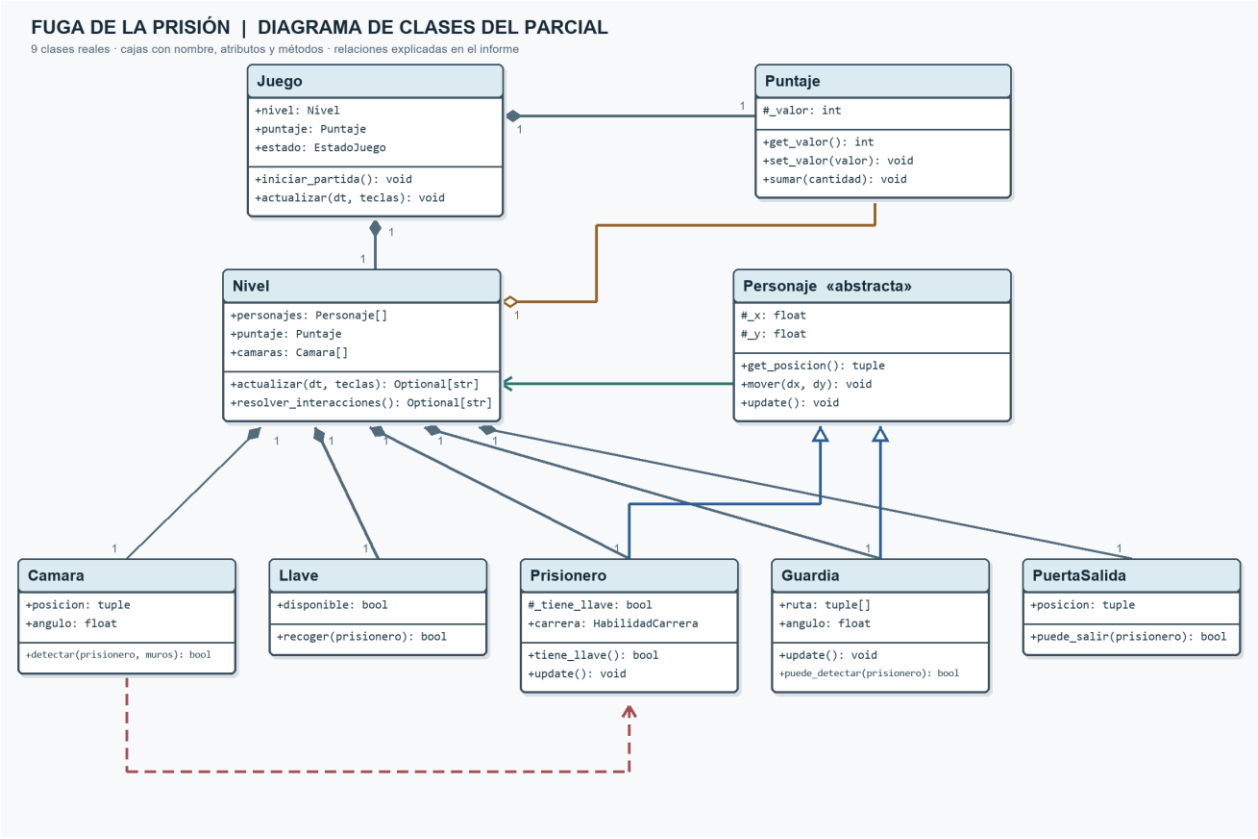


Figura 1. Nueve clases clave de las dieciocho implementadas. Fuente: modelo_parcial.mmd.

`Personaje` es la base de `Prisionero` y `Guardia`: comparten movimiento, pero actúan de forma distinta. `Juego` crea el `Nivel` y el `Puntaje`. El nivel recibe ese puntaje y crea al prisionero, al guardia, a la cámara, a la llave y a la salida. Además, cada personaje guarda una referencia a su nivel. La cámara usa al prisionero recibido por `detectar()` para comprobar si puede verlo. Estas relaciones aparecen en la figura 1.

b. Estructura de componentes

`Juego` inicia la partida y controla si está jugando, en pausa, en derrota o en victoria. `Nivel` guarda el mapa y decide cuándo se recoge la llave, se detecta al jugador o se llega a la salida. `Prisionero` responde al teclado; `Guardia` recorre su ruta; `Camara` vigila una zona. `Puntaje` lleva los puntos y avisa a `PanelPuntaje` cuando cambian. `Vista` dibuja la pantalla y a los personajes stickman. `HabilidadCarrera` controla la duración y la recarga de la carrera.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. Un personaje recibe su nivel y lo guarda con `self.nivel = nivel`. Así puede consultar los muros y el tiempo de la partida sin crear otro nivel.

Agregación. `Juego` crea un `Puntaje` y se lo entrega al nuevo nivel mediante `Nivel(self.puntaje)`. Al reiniciar, se crea otro nivel, pero se conserva el mismo objeto `Puntaje` y se reinicia su valor. Por eso el nivel usa un objeto que también existe fuera de él.

Composición. El nivel crea sus propios objetos, por ejemplo `self.prisionero = Prisionero(self, *centro(2, 11))`. Al comenzar una partida nueva se crea otro nivel con nuevos personajes, cámara, llave y salida. `Prisionero` también crea su propia `HabilidadCarrera`. En el UML, el rombo negro queda junto al objeto que crea esas partes.

b. Herencia y polimorfismo

`Prisionero` y `Guardia` heredan de `Personaje`, que reúne posición y movimiento. Ambos tienen un método `update()`: el prisionero lee las teclas y el guardia sigue su ruta. El nivel llama al mismo método sin preguntar cuál de los dos es:

```
for personaje in self.personajes:
    personaje.update()
```

Eso es polimorfismo: una llamada común produce el comportamiento propio de cada clase. La prueba de actualización comprueba que el bucle llama a los dos objetos.

c. Encapsulamiento y abstracción

`Puntaje` guarda el valor en `_valor`. Se consulta con `get_valor()` y se cambia con `set_valor()`, que rechaza valores inválidos y avisa al panel. El personaje guarda su posición en `_x` y `_y`; se consulta con `get_posicion()` y se modifica con `mover()`, que comprueba las colisiones. En Python, el guion bajo señala uso interno, aunque no impide por sí solo el acceso directo.

`Personaje` es una clase abstracta: sirve como base para el prisionero y el guardia, pero no se puede crear como personaje genérico. Su método `update()` obliga a cada clase hija a definir cómo actúa. Los fragmentos de código del anexo muestran estas decisiones.

Patrones de diseño implementados

State: `EstadoJuego` es la base de `Jugando`, `Pausa`, `Derrota` y `Victoria`. `Juego` usa un estado distinto para cada situación; en pausa deja de actualizarse el nivel. **Observer:** `Puntaje` avisa a `PanelPuntaje`, que implementa `ObservadorPuntaje`, cuando cambia el valor. Estas clases están en el código y se explican aquí; el UML selecciona las nueve clases centrales del juego.

VI. Implementación de Resultados Parciales

El prototipo tiene un pabellón jugable. Se puede mover al prisionero con WASD o flechas, correr con Shift, recoger la llave y llegar a la salida. Los muros detienen el movimiento y pueden ocultar al jugador de la vista del guardia o la cámara. El mapa obliga a entrar al cuarto de la llave, volver, esperar a cubierto y rodear los obstáculos de la derecha; la ruta directa por arriba está cerrada. La interfaz muestra el objetivo, el tiempo, los puntos y el estado de la carrera.



Figura 2. Interfaz inicial generada por Pygame en modo de verificación.

El 15 de octubre de 2026 se ejecutaron **22 pruebas automáticas, sin fallos**. También se completó una ruta con movimiento normal, sin cambiar directamente la posición del personaje: recogió la llave y terminó con 600 puntos. Se generaron capturas de inicio, pausa, victoria y derrota. Estas pruebas comprueban reglas y dibujo, pero todavía falta registrar un ensayo manual con teclado. La rúbrica pide mostrar un avance del 25 % para el nivel excelente, aunque no indica cómo calcular ese porcentaje [2, p. 1]; aquí se presentan funciones comprobables sin asignarles un porcentaje supuesto.

Git. En la revisión del 15 de octubre, el historial registrado hasta el commit `27afa05` tenía ocho commits: siete con el autor configurado `JhosFcL` y uno con `Simei`. El historial muestra uso de Git, pero esos nombres por sí solos no permiten identificar los aportes de los cuatro integrantes ni afirmar que se alcanzó la evidencia completa de trabajo en equipo.

VII. Conclusiones

Se logró un primer nivel con movimiento, vigilancia, llave, salida y puntaje. El diagrama y el código permiten explicar las seis relaciones y fundamentos de POO solicitados. También se pueden mostrar State en la pausa y Observer cuando cambian los puntos.

Una dificultad concreta fue hacer que los muros detuvieran al personaje y bloquearan la visión sin impedir que existiera una ruta de escape. Las pruebas de colisión, visión y recorrido ayudaron a comprobarlo. Separar las reglas del nivel del dibujo permite cambiar el mapa y verificar el comportamiento sin mezclar ambas partes. Queda pendiente ensayar la partida con teclado y dejar evidencia verificable del trabajo de cada integrante en Git.

VIII. Anexos

Diagrama de clases. La figura 1 presenta las nueve clases principales y sus relaciones.

Fragmento 1. Relaciones entre objetos. Estas tres instrucciones pertenecen, respectivamente, a `Personaje`, `Juego` y `Nivel`:

```
self.nivel = nivel
self.nivel = Nivel(self.puntaje)
self.prisionero = Prisionero(self, *centro(2, 11))
```

La primera guarda una referencia (asociación), la segunda entrega un puntaje que ya existe (agregación) y la tercera crea un personaje propio del nivel (composición).

Fragmento 2. Protección del puntaje. El método valida el dato antes de guardarlo y actualizar el panel:

```
def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError("El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

Fragmento 3. Actualización de personajes. El bucle de la sección IV muestra la llamada común a `update()` del prisionero y del guardia.

Material del curso

[1] *Proyecto Final POO*, ciclo 2026-2, pp. 1–3.

[2] *Rúbrica y Estructura - Proyecto Parcial*, ciclo 2026-2, pp. 1–3.

VIII. Anexos - Diagrama de clases ampliado

Figura 1. Nueve clases clave y sus relaciones. Fuente: modelo_parcial.mmd.

